

CS 452/652 Winter 2025 - Lecture 1

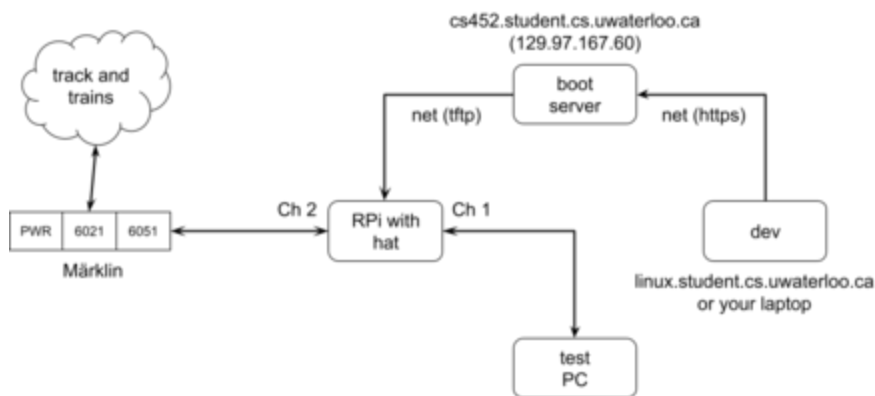
Introduction and Overview

Jan 7, 2025 [next](#)

Course Overview

- **real-time** systems
 - software that interacts with the **real** world
 - model/control physical world, communication, **timing**
 - robustness against defects (train, switch, sensor)
- **real-time systems**
 - small operating system, plus train control application
 - start from scratch, get a better understanding of things you may take for granted
 - no memory translation/protection, no file system
 - micro-kernel with send-receive-reply (SRR) communication
 - multi-threading; concurrency & non-determinism
 - device interaction; interrupt handling
- few guard rails; room to roam
 - design flexibility in assignments
 - you define and implement a project

Infrastructure



RPi Boot Process

- bootloader loads start4.elf and other files (config.txt)
- bootstraps VideoCore (VC) GPU and loads kernel8.img
- jumps to 0x80000 to hand control to kernel
- VideoCore does other things behind the scenes

Development Overview

- basic procedure
 1. build and create .img file on dev box
 2. upload .img to boot server (cs452.student.cs.uwaterloo.ca)

3. reboot / power on RPi (output to gtkterm on test PC)
4. RPi automatically loads image for its MAC address

Sample Code

- iotest
 - very simple, does console input/output using channel 1
 - use it to try the dev/test process
- development tools
 - cross-compiler and other usual tools (linker, readelf, etc)
 - on linux.student.cs.uwaterloo.ca, tools are in /u/cs452/public/xdev/bin
 - on personal computer, install your own

Development Basics

- *freestanding* code
 - no operating system, no libraries (including libc)
 - explicit linking rules
- load and execute: ELF/image files
 - at runtime, program will have code, data, stack (no heap)
 - build process creates ELF file
 - text, data, bss sections
 - start of the text section is the entry point
 - img file is produced from ELF - includes text and data (not bss)
 - bss variables need to be initialized by the program
- libc (newlib) is part of compiler package, but no system calls
 - can't rely on underlying OS to provide services, like I/O, memory allocation
- MMU is disabled
- FP/SIMD
 - kernels do not typically use FP operations, but modern compilers/libc use SIMD
 - SIMD instructions work, but unaligned accesses fail without MMU
 - `-mstrict-align` compiler flag
 - problem with precompiled libc, etc.
 - SIMD test in Makefile → `-mgeneral-regs-only` compiler flag
 - options:
 - don't use libc (like iotest)
 - use only non-problematic bits of libc (see check in iotest Makefile)
 - or?

Assignment 0

- due in one week! start now!
- polling loop, checking track state, clock, user input

```
for (;;) {
    if (c1) f1();
    if (c2) f2();
    ...
}
```